

WSTĘP DO PROGRAMOWANIA

Lista 2

1

2

3

Zadanie 1

Rozważmy następujące zapisy petli:

```
1 for (;;) { ... }
2
3 while(true) { ... }
4
5 for (++;x) { ... }
6
7 for (;i<10;i++) { ... }
8
9 while (true) { ...; x++; }
10
11 for (;(i++)<10;) { ... }
```

1. Połącz ze sobą pary petli, które zachowują się identycznie.
2. Jak można by zdefiniować petle `for` gdyby nie istniała w składni języka C?
3. Jakiego końcowego typu musi być wyrażenie zawarte po środku petli `for`?

Wskazówka do punktu 2: Zbuduj ją przy użyciu petli `while`.

Wskazówka do punktu 3: To warunek, tak samo jak `while`.

Zadanie 2

Elementy tablicy w pamięci są ustawione obok siebie. Pozwala to na nietypowe operacje na nich, przy użyciu wskaźników.

Wskaźniki to tak właściwie liczby, będące adresami. Możemy więc do nich dodawać i od nich odejmować.

Poprawna operacja zatem jest coś takiego:

```
1 int* ptr = &x;
2 ptr++;
```

Wydrukuj to oczywiście śmieci.

Ale co jeżeli będziemy mieli zapisaną tablicę, np:

```
1 int i[] = {6, 9};
2 int* ptr = i;
3 ptr++;
```

Przed inkrementacją `ptr` wskazuje na pierwszy element tablicy, czyli na `6`.

Po inkrementacji pod wskaźnikiem znajduje się `9`.

Korzystając z nabytej wiedzy, zaimplementuj algorytm mnożenia dwóch macierzy bez użycia wyrażen typu `tab[i]`.

Wskazówka: Iteracja przez tablice jest możliwa przez inkrementację wskaźnika. Dostęp do elementu jest poprzez **dereferencje wskaźnika** (`*ptr`)

Zadanie 3

Rozważmy następującą funkcję:

```
1 unsigned int magic(char* str, char ch) {
2     unsigned int count = 0;
3     for (char* curr = str; *(curr++) != '\0';)
4         count += (*curr == ch);
5
6     return count;
7 }
```

Odpowiedz na następujące pytania:

1. Co robi ta funkcja?
2. Ta funkcja **nie** jest bezpieczna. Dlaczego?
3. Co można zrobić aby sprawić, żeby ta funkcja była bezpieczna w użytku? Napisz jej bezpieczną implementację.

Wskazówka do punktu 1: Ta funkcja specjalnie jest napisana w chujowy sposób. Spróbuj ją najpierw rozłożyć na trochę czytelniejszą formę.

Wskazówka do punktu 2 i 3: To ten sam problem, który ma funkcja `gets` zawarta w bibliotece standardowej. `fgets` to jej bezpieczna implementacja.